# SEV-SNP 고급 기능

이 문서는 SEV-SNP(Secure Nested Paging)의 고급 기능에 대한 상세한 설명을 제공합니다. SEV-SNP는 기존 SEV와 SEV-ES의 보안 기능을 확장하여 더욱 강력한 보안 가상화 환경을 제공합니다.

## 목차


- SEV-SNP 개요
- SEV-SNP 고유 기능
- VMPL (VM Permission Level)
- RMP (Reverse Map Table)
- 기존 SEV 대비 개선사항
- SNP 전용 API
- 기능 사용법
- 코드 예제


## SEV-SNP 개요

SEV-SNP(Secure Nested Paging)는 AMD의 3세대 보안 가상화 기술로, Milan(3세대), Genoa(4세대), Turin(5세대) EPYC 프로세서에서 지원됩니다. SNP는 기존 SEV와 SEV-ES의 기능을 확장하여 다음과 같은 고급 보안 기능을 제공합니다:

- **강화된 메모리 보호**: RMP(Reverse Map Table)를 통한 세밀한 메모리 접근 제어
- **계층적 권한 관리**: VMPL(VM Permission Level)을 통한 다단계 권한 제어
- **원격 증명**: 하드웨어 기반의 신뢰할 수 있는 증명 메커니즘

- **향상된 무결성 검증**: 게스트 메모리와 상태의 실시간 무결성 보장

## SEV-SNP 고유 기능

### 1. 강화된 메모리 암호화

SEV-SNP는 기존 SEV의 메모리 암호화 기능을 확장하여 다음과 같은 개선사항을 제공합니다:

- **페이지 단위 암호화**: 각 메모리 페이지가 개별적으로 암호화되고 관리됩니다
- **실시간 무결성 검증**: 메모리 접근 시마다 무결성이 검증됩니다
- **메모리 재배치 방지**: RMP를 통해 메모리 페이지의 무단 재배치를 방지합니다

### 2. 하드웨어 기반 증명

SEV-SNP는 하드웨어 레벨에서 다음과 같은 증명 기능을 제공합니다:

- **증명 보고서 생성**: AMD Secure Processor가 직접 증명 보고서를 생성합니다
- **측정 체인**: 부트 과정부터 런타임까지의 전체 측정 체인을 유지합니다
- **원격 검증**: 원격 당사자가 게스트의 무결성을 검증할 수 있습니다

### 3. 동적 보안 정책

SEV-SNP는 런타임에 보안 정책을 동적으로 조정할 수 있는 기능을 제공합니다:

- **정책 업데이트**: 게스트 실행 중에도 보안 정책을 업데이트할 수 있습니다
- **권한 위임**: VMPL을 통해 권한을 계층적으로 위임할 수 있습니다
- **세밀한 접근 제어**: 페이지 단위로 접근 권한을 설정할 수 있습니다

## VMPL (VM Permission Level)

VMPL(VM Permission Level)은 SEV-SNP의 핵심 기능 중 하나로, 가상 머신 내에서 계층적 권한 관리를 제공합니다.

## VMPL 개념

VMPL은 0부터 3까지 4단계의 권한 레벨을 제공합니다:

- **VMPL 0**: 최고 권한 레벨 (하이퍼바이저)
- **VMPL 1**: 게스트 OS 커널 레벨
- **VMPL 2**: 게스트 애플리케이션 레벨
- **VMPL 3**: 최하위 권한 레벨

## VMPL 권한 마스크

각 VMPL 레벨은 다음과 같은 권한을 가질 수 있습니다:

```
bitflags! {
pub  struct VmplPerms: u8 {
/// 페이지 읽기 권한
const READ = 1;
/// 페이지 쓰기 권한
const WRITE = 1 << 1;
/// 사용자 모드 실행 권한 (CPL3)
const EXECUTE_USER = 1 << 2;
/// 슈퍼바이저 모드 실행 권한 (CPL0-2)
const EXECUTE_SUPERVISOR = 1 << 3;
}
```

```
}
```

### VMPL 사용 시나리오

1. **하이퍼바이저 격리**: VMPL 0에서 하이퍼바이저가 실행되어 전체 시스템을 관리
2. **게스트 OS 보호**: VMPL 1에서 게스트 OS가 실행되어 애플리케이션으로부터 보호
3. **애플리케이션 격리**: VMPL 2-3에서 애플리케이션들이 서로 격리되어 실행

## RMP (Reverse Map Table)

RMP(Reverse Map Table)는 SEV-SNP의 핵심 보안 메커니즘으로, 시스템의 모든 물리 메모리 페이지에 대한 메타데이터를 관리합니다.

### RMP 구조

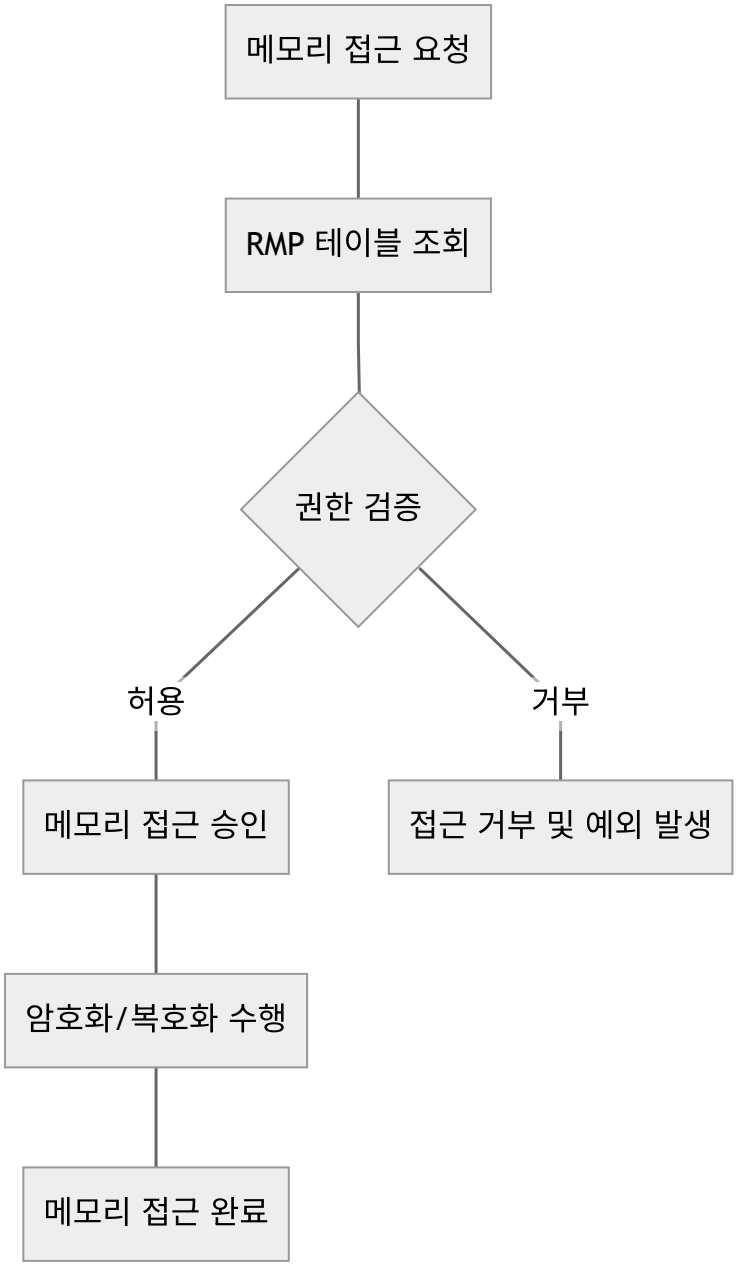
RMP는 각 물리 메모리 페이지에 대해 다음 정보를 저장합니다:

- **소유자 정보**: 해당 페이지를 소유한 게스트 VM의 식별자
- **페이지 크기**: 4KB, 2MB, 1GB 페이지 크기 정보
- **VMPL 권한**: 각 VMPL 레벨별 접근 권한
- **페이지 상태**: 유효, 무효, 펜딩 등의 상태 정보

### RMP 기능

1. **메모리 격리**: 게스트 간 메모리 접근을 하드웨어 레벨에서 차단
2. **권한 검증**: 메모리 접근 시 VMPL 권한을 자동으로 검증
3. **무결성 보장**: 페이지 메타데이터의 무단 변경을 방지

### RMP 동작 원리




## 기존 SEV 대비 개선사항

### 1. 보안 강화

| 기능 | SEV/SEV-ES | SEV-SNP |

|------|------------|---------|

| 메모리 암호화 | VM 단위 | 페이지 단위 |

| 무결성 검증 | 런치 시점 | 실시간 |

| 권한 관리 | 단일 레벨 | 다단계 (VMPL) |

| 메모리 보호 | 기본적 | 고급 (RMP) |

### 2. 성능 개선

- **하드웨어 가속**: 암호화/복호화 작업의 하드웨어 가속화
- **캐시 효율성**: 페이지 단위 암호화로 캐시 효율성 향상
- **병렬 처리**: 다중 VMPL 레벨에서의 병렬 작업 지원

### 3. 관리 편의성

- **동적 정책**: 런타임 정책 변경 지원
- **세밀한 제어**: 페이지 단위 권한 설정
- **자동화된 증명**: 하드웨어 기반 자동 증명 생성

## SNP 전용 API

SEV-SNP는 다음과 같은 전용 API를 제공합니다:

### 1. 플랫폼 관리 API

```
// SNP 플랫폼 상태 조회
pub fn snp_platform_status(&mut self) -> Result<SnpPlatformStatus, UserApiErr

// SNP 설정 커밋
pub fn snp_commit(&mut self) -> Result<(), UserApiError>

// SNP 설정 변경
```

```rust
pub fn snp_set_config(&mut self, config: Config) -> Result<(), UserApiError>


// VLEK 해시스틱 로드
pub fn snp_vlek_load(&mut self, hashstick: WrappedVlekHashstick) -> Result<()
```

## 2. 게스트 관리 API

```rust
// 증명 보고서 요청
pub fn get_report(
&mut self,
message_version: Option<u32>,
data: Option<[u8; 64]>,
vmpl: Option<u32>,
) -> Result<Vec<u8>, UserApiError>


// 확장 증명 보고서 요청
pub fn get_ext_report(
&mut self,
message_version: Option<u32>,
data: Option<[u8; 64]>,
vmpl: Option<u32>,
) -> Result<(Vec<u8>, Option<Vec<CertTableEntry>>), UserApiError>
```

```
// 파생 키 요청
pub fn get_derived_key(
&mut self,
message_version: Option<u32>,
derived_key_request: DerivedKey,
) -> Result<[u8; 32], UserApiError>
```

### 3. 런치 프로세스 API

```
// SNP 런치 시작
pub fn start(self, start: Start) -> Result<Launcher<Started, U, V>, FirmwareEr

// 데이터 업데이트
pub fn update_data(
&mut self,
update: Update,
gpa: u64,
gpa_len: u64,
) -> Result<(), FirmwareError>

// 런치 완료
pub fn finish(self, finish: Finish) -> Result<(U, V), FirmwareError>
```

# 기능 사용법

## 1. SNP 플랫폼 초기화

```rust
use sev::firmware::host::Firmware;

// SEV 펌웨어 연결
let mut firmware = Firmware::open()?;

// SNP 플랫폼 상태 확인
let status = firmware.snp_platform_status()?;
println!("SNP 플랫폼 상태: {:?}", status);

// 설정 커밋
firmware.snp_commit()?;
```

## 2. SNP 게스트 런치

```rust
use sev::launch::snp::{Launcher, Start};
use sev::firmware::guest::GuestPolicy;

// 게스트 정책 설정
let policy = GuestPolicy::default();
let gosvw = [0u8; 16];
```

```rust
let start = Start::new(policy, gosvw);

// 런처 생성 및 시작
let launcher = Launcher::new(vm_fd, sev_fd)?;
let mut launcher = launcher.start(start)?;

// 메모리 페이지 업데이트
let update = Update::new(start_gfn, data, PageType::Normal);
launcher.update_data(update, gpa, gpa_len)?;

// 런치 완료
let finish = Finish::new(None, None, [0u8; 32]);
let (vm_fd, sev_fd) = launcher.finish(finish)?;
```

### 3. 증명 보고서 생성

```rust
use sev::firmware::guest::Firmware;

// 게스트 펌웨어 연결
let mut firmware = Firmware::open()?;

// 고유 데이터 준비
let unique_data = [0u8; 64];
```

```
// 데이터에 애플리케이션별 정보 설정
// 예: 공개 키, 해시, 타임스탬프 등

// 증명 보고서 요청
let report = firmware.get_report(Some(1), Some(unique_data), Some(1))?;
println!("증명 보고서 크기: {} bytes", report.len());
```

## 코드 예제

### 예제 1: SNP 플랫폼 설정

```
use sev::{firmware::host::Firmware, Generation};

fn setup_snp_platform() -> Result<(), Box<dyn std::error::Error>> {
// 펌웨어 연결
let mut firmware = Firmware::open()?;
// 현재 세대 확인
let generation = Generation::identify_host_generation()?;
println!("검출된 프로세서 세대: {}", generation.titlecase());
// SNP 지원 여부 확인
match generation {
Generation::Milan | Generation::Genoa | Generation::Turin => {
println!("SEV-SNP 지원됨");
```

```
}
_ => {
println!("SEV-SNP 미지원");
return Ok(());
}
}
// 플랫폼 상태 조회
let  status = firmware.snp_platform_status()?;
println!("현재 TCB 버전: {:?}", status.reported_tcb);
println!("커밋된 TCB 버전: {:?}", status.committed_tcb);
// 설정 커밋 (필요한 경우)
if  status.reported_tcb != status.committed_tcb {
println!("TCB 버전 커밋 중...");
firmware.snp_commit()?;
println!("커밋 완료");
}
Ok(())
}
```

## 예제 2: 파생 키 생성

```
use sev::firmware::guest::{Firmware, DerivedKey, GuestFieldSelect};
```

```rust
fn generate_derived_key() -> Result<[u8; 32], Box<dyn std::error::Error>> {
// 게스트 펌웨어 연결
let mut firmware = Firmware::open()?;
// 파생 키 요청 생성
let derived_key_request = DerivedKey::new(
false, // VCEK 사용
GuestFieldSelect(0x01), // 게스트 필드 선택
1, // VMPL 레벨
0, // 게스트 SVN
0, // TCB 버전
None, // 런치 완화 벡터
);
// 파생 키 요청
let derived_key = firmware.get_derived_key(Some(2), derived_key_request)?;
println!("파생 키 생성 완료: {} bytes", derived_key.len());
Ok(derived_key)
}
```

### 예제 3: 페이지 타입별 메모리 업데이트

```rust
use sev::launch::snp::{Update, PageType};

fn update_guest_memory(
```

```
launcher: &mut Launcher<Started, impl AsRawFd, impl AsRawFd>,
memory_regions: Vec<(u64, &[u8], PageType)>
) -> Result<(), Box<dyn  std::error::Error>> {
for (gfn, data, page_type) in  memory_regions {
// 페이지 타입에 따른 업데이트
let  update = Update::new(gfn, data, page_type);
match  page_type {
PageType::Normal => {
println!("일반 데이터 페이지 업데이트: GFN {}", gfn);
}
PageType::Vmsa => {
println!("VMSA 페이지 업데이트: GFN {}", gfn);
}
PageType::Zero => {
println!("제로 페이지 업데이트: GFN {}", gfn);
}
PageType::Unmeasured => {
println!("측정되지 않은 페이지 업데이트: GFN {}", gfn);
}
PageType::Secrets => {
println!("시크릿 페이지 업데이트: GFN {}", gfn);
}
PageType::Cpuid => {
```

```
println!("CPUID 페이지 업데이트: GFN {}", gfn);

}

}

// 메모리 업데이트 수행

let  gpa = gfn * 4096; // 4KB 페이지 크기

let  gpa_len = data.len() as u64;

launcher.update_data(update, gpa, gpa_len)?;

}

Ok(())

}
```

이러한 고급 기능들을 통해 SEV-SNP는 기존 SEV 기술보다 훨씬 강력하고 유연한 보안 가상화 환경을 제공합니다. 특히 VMPL과 RMP를 통한 세밀한 권한 관리와 하드웨어 기반 증명 기능은 클라우드 환경에서의 신뢰할 수 있는 컴퓨팅을 가능하게 합니다.